

ALUForge

A 16-bit-instruction, 8-bit sign-magnitude processor / ALU
built gate-by-gate in Logisim-evolution

Project file	ALUforge.circ (Logisim-evolution 5.0.0)
Program ROM	program.rom (50 words, 256 x 16 ROM)
Assembly listing	program_listing.txt
Demonstration	PROJECT_DEMO.mp4 (72 s screen capture)
Circuits	19 sub-circuits, 695 components

[Complete project documentation](#)

1. Project overview

ALUForge is a small, complete processor built in Logisim-evolution. It fetches 16-bit instructions from a ROM, decodes them, and executes arithmetic on a file of five 8-bit registers. Every arithmetic block — the adder, the sign-magnitude add/subtract unit, the shift-and-add multiplier and the long-division unit — is built from gates, multiplexers and sub-circuits designed in the project itself. The register storage is built from D flip-flops made of NAND gates.

Main features

- Six instructions: **END, MOV, LOAD, SUM, MUL, DIV** (3-bit opcode).
- Five registers: **ACC, A, B, REM, RES**, each holding an 8-bit magnitude plus a separate sign bit (sign-magnitude numbers, range -255 to $+255$).
- Signed addition, multiplication and division (with remainder and divide-by-zero protection).
- Hard-wired control: a single *decode* unit generates all register write enables, so each instruction takes one clock step.
- Live LED bars for A, ACC, B, Result and Remainder, plus a cycle counter, for visual demonstration.
- A 50-instruction test program that exercises every instruction, negative operands, overflow and division by zero.

Project files

File	Description
ALUforge.circ	Logisim-evolution 5.0.0 project containing all 19 circuits. Main circuit: ALUForge .
program.rom	Logisim "v3.0 hex words plain" memory image: 50 sixteen-bit words, loaded into the ROM.
program_listing.txt	Human-readable assembly listing: address, machine word (hex) and mnemonic for each instruction.
PROJECT_DEMO.mp4	72-second screen recording (2216 x 1562) of the program running in the simulator.

2. System architecture

The top-level **ALUForge** circuit connects four groups of components: the program memory, the instruction fetch unit, the decode/execute unit and the register file.

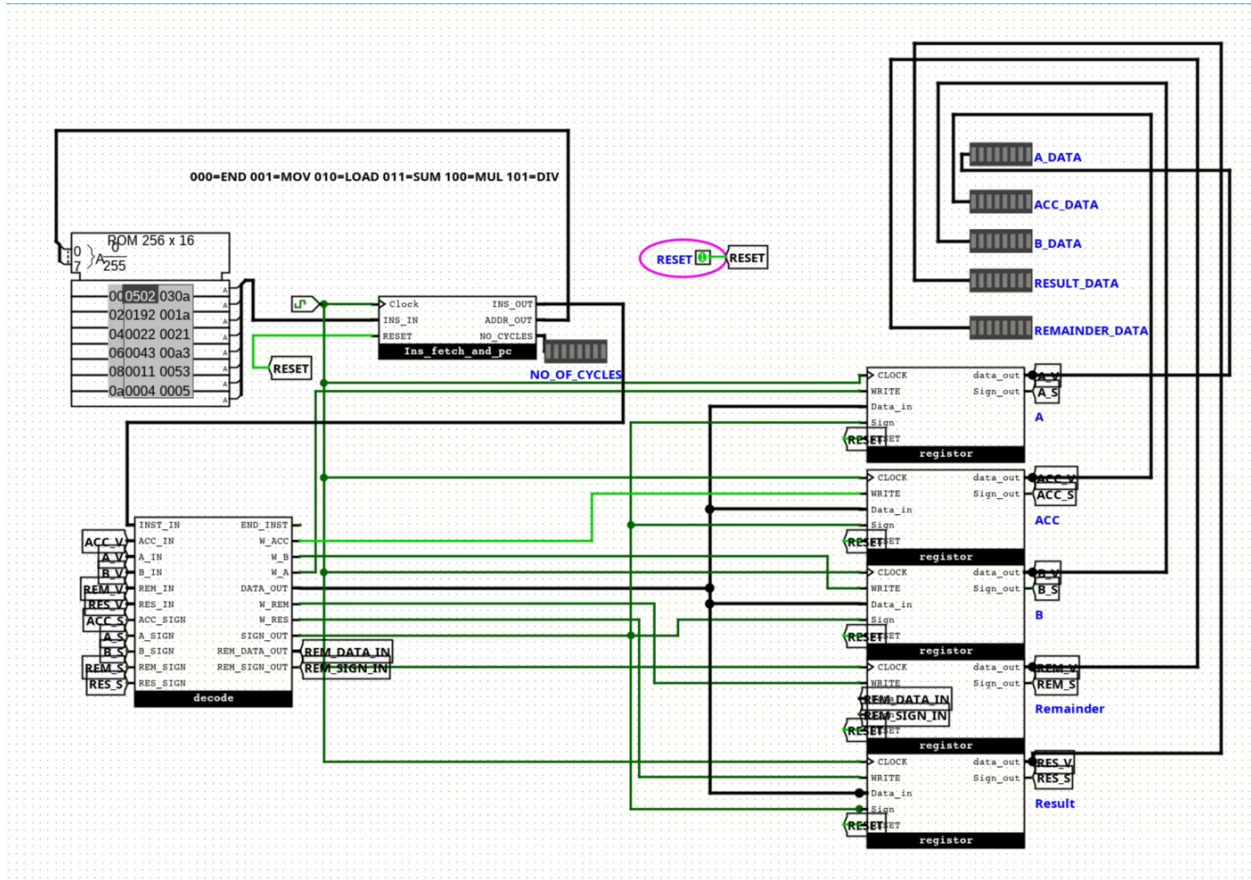


Figure 1 – Top-level ALUForge circuit as shown in the demonstration video (all registers at reset).

Block	Role
ROM (256 x 16)	Program memory. Holds the 16-bit instruction words; addressed by the program counter (8-bit address, up to 256 instructions).
Ins_fetch_and_pc	Program counter (PC register + incremter), instruction register (IR) and a TICKER cycle counter. Outputs the ROM address, the current instruction and NO_CYCLES.
decode	Control and execution unit. Splits the instruction, selects operands, contains the LOAD, Move, SUM, MULTIPLIER8 and DIV units and generates the write enables and result bus.
register x5	The register file: ACC, A, B, Remainder, Result. Each is an 8-bit data register plus a sign flip-flop.
LED bars	A_DATA, ACC_DATA, B_DATA, RESULT_DATA, REMAINDER_DATA and NO_OF_CYCLES - display register contents in binary.
Clock / RESET	A free-running clock and a RESET input that clears the registers and the program counter.

Data flow per instruction

- The PC addresses the ROM; the instruction word is latched in the IR.
- The decode unit receives the instruction plus all five register values and signs (ACC_V/S, A_V/S, B_V/S, REM_V/S, RES_V/S).
- The opcode selects which functional unit result drives the shared **DATA_OUT / SIGN_OUT** bus.

- Exactly the targeted register receives a write pulse (W_ACC, W_A, W_B, W_REM or W_RES); on the next clock edge it captures the bus. DIV additionally drives the remainder bus (REM_DATA_OUT / REM_SIGN_OUT) into REM.
- The PC advances and the next instruction is fetched. An END instruction raises END_INST, which stops the program.

3. Instruction set architecture

Every instruction is one 16-bit word. The low three bits are the opcode; the remaining bits depend on the instruction type.

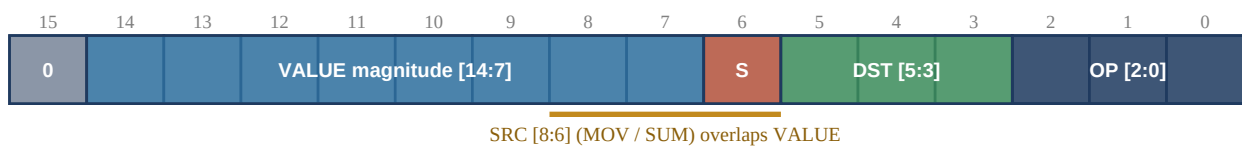


Figure 2 – Instruction word fields (bit 15 is unused and 0 in this program).

Registers and addresses

Address	Register	Purpose
000	ACC	Accumulator / general purpose
001	A	Operand A; destination of MUL and DIV
010	B	Operand B (multiplier / divisor)
011	REM	Remainder output of DIV
100	RES	Result register

Opcodes and formats

Opcode	Mnemonic	Operation	Fields
000	END	Stop program execution.	-
001	MOV d,s	Copy register s into register d.	DST = inst[5:3], SRC = inst[8:6]
010	LOAD d,#v	Load an immediate signed constant into d.	ADDR = inst[5:3], sign = inst[6], magnitude = inst[14:7]
011	SUM d,s	d = d + s (signed). Result written to the first operand (ADDR1).	ADDR1 = inst[5:3], ADDR2 = inst[8:6]
100	MUL	A = A x B (signed; low 8 bits of the product are kept).	none
101	DIV	A = A / B (truncated quotient), REM = remainder. If B = 0 the write is blocked and A/REM are unchanged.	none

Encoding examples

Word	Binary	Decoding
0502	0000 0101 0000 0010	opcode 010 = LOAD, ADDR 000 = ACC, sign 0, magnitude 0x0A = 10 → LOAD ACC,10

Word	Binary	Decoding
02ca	0000 0010 1100 1010	opcode 010 = LOAD, ADDR 001 = A, sign 1, magnitude 5 → LOAD A,-5
00a3	0000 0000 1010 0011	opcode 011 = SUM, ADDR1 100 = RES, ADDR2 010 = B → SUM RES,B
0021	0000 0000 0010 0001	opcode 001 = MOV, DST 100 = RES, SRC 000 = ACC → MOV RES,ACC
0004 / 0005	- / -	MUL / DIV (no operand fields)
0000	0000 0000 0000 0000	END

Number format

Values are stored in **sign-magnitude** form: an 8-bit unsigned magnitude (0–255) and a separate sign bit. Immediate constants therefore range from –255 to +255. Results whose magnitude exceeds 255 keep only the low 8 bits.

4. Circuit module reference

The project contains 19 circuits. They form a hierarchy from single-bit primitives up to the complete processor.

Circuit	Level	Description
ALUForge	Top	Top-level processor: ROM, fetch unit, decode unit, five registers, LED displays, RESET and clock.
Dflipflop	Primitive	Clocked D flip-flop with Write enable and RESET, built from 11 NAND, 5 NOT and 1 AND gate.
registor	Storage	8-bit register + sign bit: 9 Dflipflops with common CLOCK, WRITE and RESET. Pins Data_in, Sign, data_out, Sign_out.
IR	Storage	16-bit instruction register (two 8-bit registers).
TICKER	Control	Counter register with incrementer; counts clock cycles and drives NO_CYCLES.
Ins_fetch_and_pc	Control	Program counter, incrementer, IR and TICKER. Provides ROM address and fetched instruction.
ADD_full	Arithmetic	1-bit full adder (A, B, Cin → Sum, Cout) built from multiplexers.
ADD	Arithmetic	8-bit ripple-carry adder from eight ADD_full cells.
Rcomp	Arithmetic	Two's-complement generator (invert + add 1) for an 8-bit value.
SUM	Arithmetic	Signed sign-magnitude adder: uses Rcomp, three ADD blocks and multiplexers to add/subtract according to the operand signs; outputs magnitude and sign.
CELL	Arithmetic	Single-bit multiplier cell (AND/XOR/OR gates) with carry in/out.
STEP	Arithmetic	One partial-product row of the multiplier: eight CELLS.
MULTIPLIER8	Arithmetic	8 x 8 shift-and-add multiplier from eight STEP rows. Outputs P_HIGH, P_LOW and SIGN (SA xor SB).
Zero8	Arithmetic	8-bit zero detector (OR tree + NOR); used for divide-by-zero and loop termination.
div_circuit	Arithmetic	Combinational long-division array: eight Subtractor stages, comparator, multiplexers and Zero8 blocks. Outputs Quotient, Remainder and Error.
DIV	Arithmetic	Signed wrapper around div_circuit: quotient sign = A_sign xor B_sign; outputs Zero_Error.
LOAD	Instruction	Immediate-load unit: 3-to-8 decoder from ADDR generates the write enable for the destination register.
Move	Instruction	Register-to-register copy: multiplexer selects source, decoder selects destination.
decode	Control	Instruction decoder and control unit (235 components): opcode decoder, operand selection, per-instruction logic, output stage.

5. Decode unit in detail

The decode circuit is organised into eight labelled sections (as annotated inside the circuit):

#	Section	Function
1	Inputs	Register values and signs arrive as tunnels (ACC, A, B, REM, RES) together with the 16-bit instruction (INST_IN).
2	Opcode decoder	inst[2:0] = OPCODE is decoded (000 END, 001 MOV, 010 LOAD, 011 SUM, 100 MUL, 101 DIV); inst[15:3] = INST_HI carries operand fields.
3	MOV	DST = inst[5:3], SRC = inst[8:6]; source value and sign are selected by SRC and written to DST.
4	LOAD	ADDR = inst[5:3], VALUE = inst[14:6] (sign + 8-bit magnitude) is written into the addressed register.
5	SUM	Operands X, Y selected by ADDR1/ADDR2 = inst[5:3], inst[8:6]; result is written to ADDR1 only when the opcode is SUM.
6	MUL	A x B; the low byte is the result, sign from the operand signs.
7	DIV	A / B; remainder goes to REM. DIV_EN = OP_DIV and not divide-by-zero, so a zero divisor blocks the write.
8	Output stage	Write enables are formed first from MOV LOAD, then OR-ed with SUM / MUL / DIV; the opcode selects the result data bus and the result sign bus.

Outputs of the decode unit: **W_ACC, W_A, W_B, W_REM, W_RES** (write enables), **DATA_OUT** and **SIGN_OUT** (common result bus), **REM_DATA_OUT** and **REM_SIGN_OUT** (remainder bus) and **END_INST**.

6. Test program and expected execution trace

The ROM contains a 50-instruction program (addresses 00–49). It prepares operands with LOAD/MOV, combines them with SUM, MUL and DIV, and ends with END. The table lists each instruction with its machine word and the register contents (decimal) *after* it executes. The trace was produced by a behavioural model of the instruction set described in Section 3.

Addr	Word	Instruction	ACC	A	B	REM	RES	Note
00	0502	LOAD ACC,10	10	0	0	0	0	
01	030a	LOAD A,6	10	6	0	0	0	
02	0192	LOAD B,3	10	6	3	0	0	
03	001a	LOAD REM,0	10	6	3	0	0	
04	0022	LOAD RES,0	10	6	3	0	0	
05	0021	MOV RES,ACC	10	6	3	0	10	
06	0043	SUM ACC,A	16	6	3	0	10	
07	00a3	SUM RES,B	16	6	3	0	13	
08	0011	MOV B,ACC	16	6	16	0	13	
09	0053	SUM B,A	16	6	22	0	13	
10	0004	MUL	16	132	22	0	13	
11	0005	DIV	16	6	22	0	13	
12	0061	MOV RES,A	16	6	22	0	6	
13	00a3	SUM RES,B	16	6	22	0	28	
14	0101	MOV ACC,RES	28	6	22	0	28	

Addr	Word	Instruction	ACC	A	B	REM	RES	Note
15	02ca	LOAD A,-5	28	-5	22	0	28	
16	0112	LOAD B,2	28	-5	2	0	28	
17	0043	SUM ACC,A	23	-5	2	0	28	
18	0004	MUL	23	-10	2	0	28	
19	0061	MOV RES,A	23	-10	2	0	-10	
20	0192	LOAD B,3	23	-10	3	0	-10	
21	0005	DIV	23	-3	3	-1	-10	remainder sign modelled*
22	0119	MOV REM,RES	23	-3	3	-10	-10	
23	320a	LOAD A,100	23	100	3	-10	-10	
24	0392	LOAD B,7	23	100	7	-10	-10	
25	0005	DIV	23	14	7	2	-10	
26	0043	SUM ACC,A	37	14	7	2	-10	
27	00c3	SUM ACC,REM	39	14	7	2	-10	
28	0012	LOAD B,0	39	14	0	2	-10	
29	0005	DIV	39	14	0	2	-10	divide-by-zero: write blocked
30	0021	MOV RES,ACC	39	14	0	2	39	
31	190a	LOAD A,50	39	50	0	2	39	
32	0212	LOAD B,4	39	50	4	2	39	
33	0004	MUL	39	200	4	2	39	
34	0041	MOV ACC,A	200	200	4	2	39	
35	0292	LOAD B,5	200	200	5	2	39	
36	0005	DIV	200	40	5	0	39	
37	0043	SUM ACC,A	240	40	5	0	39	
38	0a4a	LOAD A,-20	240	-20	5	0	39	
39	0043	SUM ACC,A	220	-20	5	0	39	
40	0252	LOAD B,-4	220	-20	-4	0	39	
41	0005	DIV	220	5	-4	0	39	
42	0061	MOV RES,A	220	5	-4	0	5	
43	008a	LOAD A,1	220	1	-4	0	5	
44	0092	LOAD B,1	220	1	1	0	5	
45	008b	SUM A,B	220	2	1	0	5	
46	0041	MOV ACC,A	2	2	1	0	5	
47	03a2	LOAD RES,7	2	2	1	0	7	
48	0023	SUM RES,ACC	2	2	1	0	9	
49	0000	END	2	2	1	0	9	

* The sign shown for a negative-dividend remainder (address 21) follows the model (remainder takes the dividend's sign) and was not confirmed in the video.

Final state after END: ACC = 2, A = 2, B = 1, REM = 0, RES = 9.

Highlights of what the program tests

- **Addresses 00–14:** basic LOAD, MOV and SUM; MUL ($6 \times 22 = 132$) followed by DIV ($132 / 22 = 6$, remainder 0).
- **Addresses 15–22:** negative operands: $-5 + 28 = 23$, then $-5 \times 2 = -10$ and $-10 / 3$.
- **Addresses 23–27:** division with a non-zero remainder: $100 / 7 = 14$ remainder 2, then SUM with both quotient and REM.
- **Addresses 28–30:** divide by zero - A and REM stay at 14 and 2 because the write is blocked.
- **Addresses 31–42:** larger magnitudes ($50 \times 4 = 200$, $200 / 5 = 40$, $200 + 40 = 240$) and signed division ($-20 / -4 = 5$).
- **Addresses 43–49:** small SUM / MOV / LOAD checks and END.

7. Demonstration video and verification

PROJECT_DEMO.mp4 shows the program running in Logisim-evolution (72 seconds, 2216 x 1562). Register LED bars were read from frames at several timestamps and compared against the expected trace. In all cases they matched:

Video time	LED reading (A / ACC / B / RES / REM)	Matches trace after
~20 s	6 / 16 / 16 / 13 / 0	address 08 (MOV B,ACC)
~40 s	10 / 23 / 2 / 28 / 0	address 18 (MUL; magnitude of -10 shown)
~64 s	14 / 39 / 0 / 39 / 2	address 30 (after the blocked divide-by-zero)
~71 s	200 / 39 / 4 / 39 / 2	address 33 (MUL, $50 \times 4 = 200$)

The recording ends before the program reaches END, so the final values (addresses 34–49) come from the behavioural model rather than from the video. The LED bars display magnitudes only; signs are carried by the separate sign lines.

8. How to run the project

- Install **Logisim-evolution v5.0.0** (the project was saved with this version; other versions may warn on load).
- Open **ALUforge.circ** and select the **ALUForge** circuit.
- The ROM already contains the program. To reload it: right-click the ROM → Edit Contents → Open, and choose **program.rom**.
- Pulse RESET (set to 1, then back to 0) to clear the registers and PC.
- Enable the clock (Simulate → Ticks Enabled, or press Ctrl+K) and watch the LED bars. Use Ctrl+T to single-step.

Writing a new program

Assemble each instruction into a 16-bit word using Section 3, write the words in hex in program.rom (the first line must be `v3.0 hex words plain`) and end the program with 0000 (END). Example: `LOAD B,3 = (3 << 7) | (0 << 6) | (010 << 3) | 010 = 0x180 + 0x10 + 0x02 = 0x0192`.

9. Limitations and possible extensions

- No jump / branch or comparison instructions: programs are straight-line and at most 256 words.
- 8-bit magnitudes: results above 255 wrap (MUL keeps only the low byte; the high byte P_HIGH is available inside the multiplier but unused).
- MUL and DIV always use A and B; MUL/DIV results go to A (DIV also to REM).
- No overflow or carry flags.
- Possible extensions: conditional jumps, a status register, wider data path, SUB/AND/OR/shift instructions, RAM load/store.